

Code Walkthrough

DNF 刷图代码逐段讲解版

按照“上面放代码，下面解释作用、执行逻辑、它是干什么的”这种方式，把主刷图代码从头到尾拆开讲

工作区： `D:\yolo\yolo11\dnf`

重点文件： `main.py`、`detector.py`、`door_strategy.py`、`minimap_astar.py`、`minimap_nav.py`、`game.py`

补充说明： `minimap_route_demo.py` 会讲，`olaplug` 只做附录说明，因为它不是当前主刷图流程里真正执行的代码。

这份文档不是“概括介绍”，而是“按代码段落顺序往下讲”。你看到的每一段，都会先把原代码摆出来，再紧跟解释它的作用、执行逻辑、以及它在整套刷图链路中的位置。这样你就能像看老师做代码批注一样，一段一段往下读。

先提醒一个非常关键的事实：当前这套刷图主线没有真正调用 `olaplug` 里的 `BindWindow()`。它实际运行依赖的是 `win32gui` 找窗口、`pyautogui` 截图、`pydirectinput` 发键盘输入。所以后面的重点会放在“现在真实运行的主链路”上。

阅读顺序

1. `main.py`：程序总入口
2. `detector.py`：大画面目标检测
3. `door_strategy.py`：门选择评分
4. `minimap_astar.py`：A 星算法
5. `minimap_nav.py`：小地图识别与路线生成
6. `game.py`：打怪、拾取、移动、卡住恢复
7. `minimap_route_demo.py`：单独演示小地图导航的测试脚本
8. 附录：为什么 `olaplug` 不属于当前主刷图执行链

第一章: `main.py`

这个文件负责什么： 它是总入口。你可以把它看成“总调度器”。它负责找窗口、截图、调用识别器、调用小地图导航器，再把结果交给动作控制器 `Game` 去执行。

片段 1: 文件头、导入、路径兼容、窗口常量

main.py 顶部

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
@File      : main.py
@Desc      : Win32-based DNF capture loop
"""

from __future__ import annotations

import pathlib
import time
from typing import Optional, Tuple

import cv2
import numpy as np
import pyautogui
import win32con
import win32gui
from loguru import logger

from dnf.game import Game
from dnf.minimap_nav import MiniMapNavigator

temp = pathlib.PosixPath
pathlib.PosixPath = pathlib.WindowsPath

from dnf.detector import Detector

WINDOW_OFFSET_X = 10
WINDOW_OFFSET_Y = 10
WINDOW_TITLE_KEYWORD = "鎧領筊錯疼笄鐸囩+"
WINDOW_CLASS_NAMES = {"鎧領筊錯疼笄鐸囩+", "鎧領筊錯疼笄鐸囩+錄淚終涓涓�邯"}
```

这一段的作用

这段代码先把整个主程序需要的库全部导进来。`cv2` 用来处理图像，`numpy` 用来承接截图数据，`pyautogui` 用来截图，`win32gui` 和 `win32con` 用来找和摆放游戏窗口，`Game` 负责动作执行，`MiniMapNavigator` 负责小地图导航，`Detector` 负责 YOLO 检测。

执行逻辑

`temp = pathlib.PosixPath` 和后面的 `pathlib.PosixPath = pathlib.WindowsPath` 是一个兼容性小技巧，通常是为了防止某些模型或依赖在 Windows 下加载路径时，把路径对象当成了 Posix 风格。然后定义的几个窗口常量，是后面找 DNF 窗口和摆窗口位置时要用的“过滤条件”和“摆放目标位置”。

它在整套程序里的位置

这是总入口文件的“基础设施区”。还没开始真正刷图，但后面所有逻辑都会依赖这里导入的类和常量。你可以理解为程序开机后的准备工作。

片段 2: 寻找 DNF 窗口

函数 `_find_dnf_window()`

```
def _find_dnf_window() -> int:
    candidates = []

    def _enum_windows(hwnd: int, _: object) -> None:
        if not win32gui.IsWindowVisible(hwnd):
            return

        title = win32gui.GetWindowText(hwnd).strip()
        class_name = win32gui.GetClassName(hwnd).strip()
        if not title and not class_name:
            return

        title_match = WINDOW_TITLE_KEYWORD in title
        class_match = class_name in WINDOW_CLASS_NAMES or WINDOW_TITLE_KEYWORD in
class_name
        if not title_match and not class_match:
            return

        try:
            left, top, right, bottom = win32gui.GetWindowRect(hwnd)
            area = max(0, right - left) * max(0, bottom - top)
        except win32gui.error:
            area = 0

        candidates.append((area, hwnd, title, class_name))

    win32gui.EnumWindows(_enum_windows, None)
    if not candidates:
        raise RuntimeError("婁℃沿錄惧垠 DNF 给耦彝")

    candidates.sort(key=lambda item: item[0], reverse=True)
    hwnd = candidates[0][1]
    logger.info(
        "緇戔畚给耦彝: hwnd={}, title={}, class={}",
        hwnd,
        candidates[0][2],
        candidates[0][3],
    )
    return hwnd
```

这一段的作用

这段代码的任务只有一个：在系统所有可见窗口里，把 DNF 窗口找出来，并返回它的句柄 `hwnd`。句柄你可以理解成“这个窗口在 Windows 系统里的身份证号”。

执行逻辑

程序先创建一个空列表 `candidates`，准备装所有疑似 DNF 的窗口。然后调用 `win32gui.EnumWindows()` 遍历系统里所有窗口，每遍历到一个窗口，就让内部函数 `_enum_windows()` 去检查。检查顺序是：先看它是否可见，再拿到窗口标题和类名；如果标题和类名都空，就跳过；如果标题和类名都不匹配 DNF 关键词，也跳过。只有疑似 DNF 的窗口才继续往下。接着它会读出窗口矩形大小，算一个面积，把面积、句柄、标题、类名一起存入列表。

它是干什么的

它的本质不是“后台绑定窗口”，而是“从系统里找出我要盯着哪一个游戏窗口”。最后代码按面积从大到小排序，选最大的那个候选窗口作为最终结果，这样可以避免同时开多个相关窗口时误选到很小的子窗口。

片段 3：把窗口摆到固定位置

函数 `_place_window()`

```
def _place_window(hwnd: int) -> None:
    win32gui.ShowWindow(hwnd, win32con.SW_RESTORE)
    left, top, right, bottom = win32gui.GetWindowRect(hwnd)
    width = max(1, right - left)
    height = max(1, bottom - top)
    win32gui.SetWindowPos(
        hwnd,
        win32con.HWND_TOPMOST,
        WINDOW_OFFSET_X,
        WINDOW_OFFSET_Y,
        width,
        height,
        win32con.SWP_SHOWWINDOW,
    )
```

这一段的作用

找到窗口之后，程序会把窗口恢复出来并移动到一个比较固定的位置，也就是屏幕左上附近的 `(10, 10)`。

执行逻辑

`ShowWindow(..., SW_RESTORE)` 的意思是如果窗口最小化了，就把它恢复。然后读取窗口当前的四个边界坐标，算出宽高。最后调用 `SetWindowPos()` 把窗口放到预设位置，并设置成置顶显示。

它是干什么的

这一步的目的，是让后面的截图区域更稳定。因为程序后面截图时依赖窗口客户区的位置，如果窗口乱跑、被挡住、缩到任务栏里，检测结果就会变得不稳定。

片段 4：计算客户区与截图

函数 `_get_client_region()` 与 `_capture_client()`

```
def _get_client_region(hwnd: int) -> Tuple[int, int, int, int]:
    left_top = win32gui.ClientToScreen(hwnd, (0, 0))
    client_rect = win32gui.GetClientRect(hwnd)
    left = int(left_top[0])
    top = int(left_top[1])
    width = int(client_rect[2] - client_rect[0])
    height = int(client_rect[3] - client_rect[1])
    if width <= 0 or height <= 0:
        raise RuntimeError("鑾峰彌媿告堯淪(=)垲鍤哄け璐?")
    return left, top, width, height

def _capture_client(hwnd: int) -> Tuple[np.ndarray, Tuple[int, int]]:
    left, top, width, height = _get_client_region(hwnd)
    img = pyautogui.screenshot(region=(left, top, width, height))
    img_np = np.array(img)
    return img_np, (width, height)
```

这一段的作用

这段代码负责真正把游戏画面“拍下来”。注意它截的是窗口客户区，也就是游戏真正显示内容的区域，而不是包括标题栏和边框的整个窗口。

执行逻辑

`ClientToScreen(hwnd, (0, 0))` 会把客户区左上角换算成屏幕坐标，`GetClientRect()` 会返回客户区的宽高。两个结果组合起来，就能得到截图需要的 `left, top, width, height`。然后 `pyautogui.screenshot(region=...)` 用这个矩形去截屏，截出来的是一个 PIL 图片，再转成 `numpy` 数组，后面才能送给 OpenCV 和 YOLO。

它是干什么的

这一段就是“给程序眼睛喂画面”。如果没有截图，后面的检测器、导航器、动作判断就全部无从谈起。

片段 5：主函数初始化

函数 `main()` 的开头

```
def main() -> None:
    device_type = ""
    detector = Detector(device_type)
    navigator = MiniMapNavigator("generic")

    hwnd = _find_dnf_window()
    _place_window(hwnd)
```

这一段的作用

主函数刚开始时，会先把核心工具对象都准备好。`Detector` 是大画面检测器，`MiniMapNavigator` 是小地图导航器。然后它立刻去找 DNF 窗口并摆好位置。

执行逻辑

`device_type = ""` 代表让检测器自己判断使用 GPU 还是 CPU。接着构造 `Detector` 和 `MiniMapNavigator("generic")`。这表示当前默认按 `generic` 地图规格来读小地图。准备好模型和导航器后，程序立刻绑定到目标窗口并把窗口移动到固定位置。

它是干什么的

这是一轮大循环开始前的“一次性初始化”。模型只加载一次，窗口只摆一次，后面每一帧只做截图、检测、判断和执行，不会反复重复这些重准备动作。

片段 6：主循环中的截图、识别和导航

函数 `main()` 的核心前半段

```
while True:
    start_time = time.time()
    try:
        img_np, (width, height) = _capture_client(hwnd)
    except Exception as exc:
        logger.exception("贴图识别失败: {}", exc)
        time.sleep(0.2)
        continue

    img, obj = detector.detect(img_np)
    logger.info("obj: {}", obj)

    frame_bgr = cv2.cvtColor(img_np, cv2.COLOR_RGB2BGR)
    route_snapshot = navigator.build_route_snapshot(frame_bgr, obj or [])
    route_direction = (
        route_snapshot.next_room_direction.upper()
        if route_snapshot.next_room_direction
        else None
    )
```

这一段的作用

这是真正每一帧都在重复执行的核心前半段。它先截图，然后做大画面识别，再做小地图导航，最后拿到一个“下一步应该往哪边走”的方向提示。

执行逻辑

程序进入死循环后，先记录开始时间，方便统计每一轮耗时。接着调用 `_capture_client(hwnd)` 截一张当前游戏画面。如果截图失败，就记录异常，稍微休眠一下，直接开始下一轮。截图成功后，把图像送进 `detector.detect()`。这个函数会返回两样东西：一张画好框的图 `img`，以及检测对象列表 `obj`。然后它把原始截图从 RGB 转成 BGR，送进 `navigator.build_route_snapshot()`。导航器会综合小地图模板匹配结果和目标检测结果，返回一个路线快照对象，里面有当前房间、目标房间、路线方向等信息。最后从里面取出 `next_room_direction`，并转成大写形式给后面的动作层用。

它是干什么的

你可以把这一段理解成“看图 + 算方向”。它本身不按键，也不打怪，只负责得出一个结论：现在画面里有哪些东西，当前副本路线应该往哪个房间走。

片段 7: 把方向交给动作层执行

函数 `main()` 的核心后半段

```
    logger.info(
        "route: current={}, boss={}, query={}, elite={}, down={}, direction={},
door={}, scores={}",
        route_snapshot.current_room,
        route_snapshot.boss_room,
        route_snapshot.query_room,
        route_snapshot.elite_room,
        route_snapshot.down_room,
        route_direction,
        route_snapshot.selected_door_center,
        route_snapshot.debug_scores,
    )

    game = Game(obj, width, height, route_direction)
    game.run()

    display = cv2.resize(img, (640, 360))
    cv2.imshow("112233", display)
    logger.info("澶勫姳鎬堝墮椹: {}", time.time() - start_time)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

这一段的作用

这里是主循环的后半段：把刚刚得到的识别结果和路线方向交给动作执行器 `Game`，让它决定这一帧到底该打怪、捡东西还是往门走。

执行逻辑

先打印一大串路线调试信息，方便开发时看当前识别到的房间和方向。然后创建 `Game(obj, width, height, route_direction)`，把检测对象、画面宽高和房间级方向一起交给它。接着调用 `game.run()` 执行动作。执行完后，会把检测结果图缩放显示到一个窗口里，便于肉眼观察程序识别得对不对。最后记录本轮处理耗时，并允许用户按 `q` 退出。

它是干什么的

这段就是整个主链路的“分发和收尾”：上游已经看懂画面，下游要把这个判断变成键盘动作。可以说 `main.py` 到这里就完成了自己的职责，后面真正的行为控制就交给 `Game` 了。

第二章：detector.py

这个文件负责什么： 用 YOLO 模型识别大画面里的玩家、怪物、Boss、门、金币、掉落物。它相当于主程序的第一只眼睛。

片段 1：检测器初始化

类 `Detector` 的 `__init__`

```
class Detector:
    def __init__(self, device_type: str = ""):
        self.img_size = 640
        self.conf_thres = 0.75
        self.iou_thres = 0.45
        self.hide_labels = False
        self.hide_conf = False

        self.device = device_type or ("0" if torch.cuda.is_available() else "cpu")
        self.weights = str(Path(__file__).resolve().parent / "shzn.pt")
        self.model = YOLO(self.weights)
        self.names = self.model.names
        self.colors = [[random.randint(0, 255) for _ in range(3)] for _ in
                        range(len(self.names))]
```

这一段的作用

这段代码负责把 YOLO 检测器真正准备好。它会设置图像大小、置信度阈值、IOU 阈值，然后加载本地模型文件 `shzn.pt`。

执行逻辑

`img_size = 640` 表示推理时把图像按 640 尺寸送进模型。`conf_thres = 0.75` 表示低于 0.75 置信度的目标就不要了。`iou_thres = 0.45` 用来做 NMS 去重。设备选择逻辑是：如果外面没强制指定设备，就自动判断显卡是否可用，可用就用 GPU 0，不可用就退到 CPU。之后用 `YOLO(self.weights)` 把模型载入内存，再从模型里拿到类别名称表 `self.names`，最后给每个类别随机生成一个画框颜色。

它是干什么的

这就是目标检测器的“开机步骤”。如果这一段没跑通，程序后面就根本不知道画面里哪里是怪、哪里是门、哪里是玩家。

片段 2：标签标准化与坐标格式转换

两个静态工具函数

```
@staticmethod
def _normalize_label(label: str) -> str:
    label = str(label).strip().lower()
    if "door" in label:
        return "door"
    return label

@staticmethod
def _xyxy_to_xywh_top_left(xyxy: np.ndarray) -> List[float]:
    x1, y1, x2, y2 = [float(v) for v in xyxy]
    return [x1, y1, max(0.0, x2 - x1), max(0.0, y2 - y1)]
```

这一段的作用

这一段不是做检测，而是负责把模型吐出来的数据整理成后面更容易用的格式。

执行逻辑

`_normalize_label()` 会把类别名统一转成小写字符串。如果类别名字里包含 `door`，就统统标准化成 `door`。这样就算模型里存在不同门标签，后面动作层也可以统一按“门”处理。

`_xyxy_to_xywh_top_left()` 则把 YOLO 常用的矩形框格式 `[x1, y1, x2, y2]` 转成 `[x, y, w, h]`。这里的 `x, y` 是左上角，`w, h` 是宽高。

它是干什么的

本质上是在做“数据清洗和格式统一”。后面的 `Game`、导航器、门策略都依赖这种统一格式，否则每个模块都得自己重新解析框数据。

片段 3：检测函数的入口与空值保护

函数 `detect()` 的开头

```
def detect(self, img0: np.ndarray) -> Tuple[np.ndarray | None, list | None]:
    """
    鍬剧塲榼勳緘鉅?          杓旂誤:
    - annotated_bgr: 鑿悔ソ矜嘮殒 BGR 鍬惧儇
    - obj: [{'monster': {'xywh': [x, y, w, h], 'conf': 0.84}}, ...]
    """
    if img0 is None:
        return None, None

    img_bgr = cv2.cvtColor(img0, cv2.COLOR_RGB2BGR)
```

这一段的作用

这段代码是目标检测的真正入口。它接收一张截图，准备把它送进 YOLO 模型。

执行逻辑

第一步先判断输入图像是不是空，如果是空图，就直接返回空结果。这样可以避免程序某一帧截图失败时崩掉。然后它把图片从 RGB 转成 BGR。原因是前面的截图来自 `pyautogui`，通常是 RGB，而 OpenCV 和后面的一些显示逻辑习惯用 BGR。

它是干什么的

它相当于把外部输入的原始截图，转换成适合 YOLO 和 OpenCV 联合处理的图像格式，为真正的模型推理做准备。

片段 4：调用 YOLO 做推理

函数 `detect()` 的模型预测部分

```
result = self.model.predict(
    source=img_bgr,
    imsz=self.img_size,
    conf=self.conf_thres,
    iou=self.iou_thres,
    device=self.device,
    verbose=False,
)[0]

annotator = Annotator(img_bgr.copy(), line_width=3, example=self.names)
obj = []
```

这一段的作用

这段代码真正调用了模型，让模型对当前这一帧做目标检测。

执行逻辑

`self.model.predict(...)` 会返回一个结果列表，因为这里一次只推一张图，所以取第一个结果 `[0]`。传进去的参数正是初始化时配置好的图像大小、置信度阈值、IOU 阈值和设备。

`Annotator` 则是一个画框辅助工具，它会把每个检测框和类别名画到图上，方便显示调试。

`obj = []` 是准备收集后面整理好的结构化检测结果。

它是干什么的

从这一行开始，程序已经不是在“准备检测”，而是在“真正看图”。从这一步开始，程序会知道当前画面里到底有没有玩家、怪物、门和掉落物。

片段 5：读取检测框并整理输出

函数 `detect()` 的后半段

```
if result.bboxes is not None and len(result.bboxes) > 0:
    boxes_xyxy = result.bboxes.xyxy.cpu().numpy()
    boxes_conf = result.bboxes.conf.cpu().numpy()
    boxes_cls = result.bboxes.cls.cpu().numpy().astype(int)

    for xyxy, conf, cls in zip(boxes_xyxy, boxes_conf, boxes_cls):
        raw_label = self.names[int(cls)]
        label = self._normalize_label(raw_label)
        xywh = self._xyxy_to_xywh_top_left(xyxy)
        conf = round(float(conf), 2)

        annotator.box_label(
            xyxy,
            raw_label if self.hide_conf else f"{raw_label} {conf:.2f}",
            color=tuple(self.colors[int(cls)]),
        )
        obj.append({label: {"xywh": xywh, "conf": conf}})

    return annotator.result(), obj
```

这一段的作用

这段代码把模型原始输出整理成后面各个模块更好消费的格式，并且顺手画框形成调试图。

执行逻辑

首先判断有没有检测框。如果有，就把每个框的位置、置信度、类别编号都取出来，并从 GPU 张量转成 CPU 上的 `numpy` 数组。然后逐个框处理：先拿类别编号去类别名表里找出原始标签，再做标准化，再把坐标格式从 `xyxy` 转成 `xywh`，最后把置信度保留两位小数。

`annotator.box_label()` 用来把这个框画到图上，而 `obj.append({label: {"xywh": xywh, "conf": conf}})` 则把它存成统一格式，供后面的逻辑使用。

它是干什么的

这一段的最终成果有两样：一张可视化后的图，一份结构化对象列表。前者给人看，后者给程序做决策用。主刷图逻辑真正依赖的是这个 `obj` 列表。

第三章： `door_strategy.py`

这个文件负责什么： 当 A 星告诉程序“下一间房在右边”时，这个文件负责在当前画面检测到的多扇门里，挑出最适合往右走的那一扇。

片段 1：门的数据结构和方向反转

文件顶部

```
from dataclasses import dataclass
from typing import List, Optional, Sequence, Tuple

Point = Tuple[float, float]

@dataclass
class DoorCandidate:
    bbox: Sequence[float]
    center: Point

def opposite_direction(direction: Optional[str]) -> Optional[str]:
    mapping = {
        "left": "right",
        "right": "left",
        "up": "down",
        "down": "up",
    }
    return mapping.get(direction)
```

这一段的作用

这一段先定义门候选的数据结构，再定义一个方向取反函数。门候选里面只存两样最关键的数据：包围框和中心点。

执行逻辑

`DoorCandidate` 是一个简单的数据类，后面导航器会把所有检测到的门包装成这种对象。
`opposite_direction()` 则用来计算某个方向的反方向，比如往左的反方向是右，往上的反方向是下。

它是干什么的

这个文件后面的评分逻辑，需要知道上一轮的方向和它的反方向，好在挑门时减少“掉头回走”的概率。所以这里先把基础工具定义出来。

片段 2：计算门相对玩家的偏移

函数 `_axis_offsets()` 与 `_direction_match()`

```
def _axis_offsets(player: Point, door: Point) -> Tuple[float, float]:
    dx = door[0] - player[0]
    dy = door[1] - player[1]
    return dx, dy

def _direction_match(expected_direction: str, dx: float, dy: float, margin: float) -> bool:
    if expected_direction == "right":
        return dx > margin
    if expected_direction == "left":
        return dx < -margin
    if expected_direction == "up":
        return dy < -margin
    if expected_direction == "down":
        return dy > margin
    return True
```

这一段的作用

这段代码负责回答两个问题：第一，某扇门相对玩家偏在哪边；第二，这扇门算不算符合期望方向。

执行逻辑

`_axis_offsets()` 很简单，就是拿门中心减去玩家中心，得到 `dx` 和 `dy`。然后 `_direction_match()` 根据期望方向去判断。例如期望往右走时，只有 `dx > margin` 才算真正符合“右门”，也就是门必须明显在玩家右边，不是勉强靠右一点点。

它是干什么的

这里的 `margin` 就是一种容差。因为门的位置不可能永远刚好标准对齐，如果不留这个余量，就容易把斜方向或挨得很近的门误判成目标门。

片段 3：无门和无方向时怎么处理

函数 `choose_best_door()` 的前半段

```
def choose_best_door(
    doors: Sequence[DoorCandidate],
    player_center: Point,
    expected_direction: Optional[str],
    last_direction: Optional[str] = None,
    margin: float = 16.0,
) -> Optional[DoorCandidate]:
    if not doors:
        return None
    if expected_direction is None:
        return min(doors, key=lambda item: abs(item.center[0] - player_center[0]) +
abs(item.center[1] - player_center[1]))
```

这一段的作用

这段是门选择函数的兜底起步逻辑。先处理一些简单情况，避免后面复杂评分没必要地执行。

执行逻辑

如果门列表为空，函数直接返回 `None`，表示当前没有可选门。如果有门但外部没有给出期望方向，那它就退而求其次，直接选离玩家最近的门。这里用的是曼哈顿距离形式的近似值，也就是 `abs(dx) + abs(dy)`。

它是干什么的

它保证了门策略在“没有路线提示”时也还能工作。也就是说，哪怕 A 星暂时没算出方向，程序也不会完全失去行动能力。

片段 4: 核心评分循环

函数 `choose_best_door()` 的主循环

```
reverse_direction = opposite_direction(last_direction)
candidates: List[Tuple[Tuple[float, float, float], DoorCandidate]] = []
fallback: List[Tuple[Tuple[float, float, float], DoorCandidate]] = []

for door in doors:
    dx, dy = _axis_offsets(player_center, door.center)
    abs_dx = abs(dx)
    abs_dy = abs(dy)
    match = _direction_match(expected_direction, dx, dy, margin)

    if expected_direction in ("left", "right"):
        forward = dx if expected_direction == "right" else -dx
        sideways = abs_dy
    else:
        forward = dy if expected_direction == "down" else -dy
        sideways = abs_dx

    reverse_penalty = 0.0
    if reverse_direction and expected_direction == reverse_direction:
        reverse_penalty = 2000.0

    score = (-forward + reverse_penalty, sideways, abs_dx + abs_dy)
    if match:
        candidates.append((score, door))
    fallback.append((score, door))
```

这一段的作用

这是整份门策略里最关键的地方。它会给每一扇门打分，然后区分成“方向匹配候选”和“普通兜底候选”两类。

执行逻辑

程序先算出上一方向的反方向，作为后面反向惩罚的依据。然后对每扇门，计算门相对玩家的横纵偏移。`match` 表示这扇门是否满足期望方向，例如往右时门是否明显在右边。如果期望方向是左右，那么 `forward` 就看左右前进量，`sideways` 看上下偏移；如果期望方向是上下，则相反。之后它还会根据上一方向的反方向，加一个非常大的 `reverse_penalty`，用来减少程序刚刚往左，下一瞬间又马上掉头往右的情况。最终分数 `score` 是一个三元组：第一优先看前进值，第二优先看侧向偏移，第三优先看总距离。

它是干什么的

本质上这是一个启发式评分算法，不是深度学习。它的目标是把“房间级方向”翻译成“画面级门选择”，也就是把 A 星给出的抽象结果变成现实可走的那扇门。

片段 5：从候选中挑出最终的门

函数 `choose_best_door()` 的结尾

```
if candidates:
    candidates.sort(key=lambda item: item[0])
    return candidates[0][1]

fallback.sort(key=lambda item: item[0])
return fallback[0][1]
```

这一段的作用

这段代码负责从前面打好分的门里，真正选出最后那一扇。

执行逻辑

如果有“方向匹配”的候选门，就按分数排序，拿分数最优的那扇。如果一个都没有，说明当前画面里没有明显满足目标方向的门，那就退回到 `fallback` 列表里，从所有门里选最合理的一扇。

它是干什么的

这是门策略的最终决策出口。它保证了“优先尊重路线方向，但找不到完全匹配的门时，也要尽量选一个最像的门”，从而避免程序卡死在门选择这一步。

第四章： `minimap_astar.py`

这个文件负责什么： 它负责房间级路径规划。注意是“房间级”，不是“像素级”。它的输出不是角色脚下怎么一步一步拐弯，而是“下一间房应该往上、下、左、右哪边走”。

片段 1：节点结构与启发函数

文件顶部

```
import heapq
from dataclasses import dataclass
from typing import Iterable, List, Optional, Sequence, Tuple

GridPoint = Tuple[int, int]

@dataclass(order=True)
class _Node:
    f: int
    position: GridPoint
    g: int = 0
    h: int = 0
    parent: Optional["_Node"] = None

def heuristic(a: GridPoint, b: GridPoint) -> int:
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

这一段的作用

这段定义了 A 星算法最基础的数据结构：节点，以及估价函数 `heuristic()`。

执行逻辑

`_Node` 里有五个核心字段：`position` 是当前格子坐标，`g` 是从起点走到这里的真实代价，`h` 是从这里到终点的估计代价，`f` 是总估价，也就是 `g + h`，`parent` 是当前节点的父节点，后面回溯路径时要用。`heuristic()` 用的是曼哈顿距离，因为当前地图只考虑上下左右四联通，不考虑斜走。

它是干什么的

这是 A 星算法的地基。没有节点结构，就没法维护路径搜索状态；没有启发函数，A 星也就退化成普通的广搜或 Dijkstra 风格搜索了。

片段 2：获取邻居节点

函数 `_neighbors()`

```
def _neighbors(position: GridPoint, grid: Sequence[Sequence[int]], priority: str) ->
    Iterable[GridPoint]:
    x, y = position
    directions = {
        "right": ((0, 1), (1, 0), (-1, 0), (0, -1)),
        "left": ((0, -1), (-1, 0), (1, 0), (0, 1)),
        "up": ((-1, 0), (0, -1), (0, 1), (1, 0)),
        "down": ((1, 0), (0, 1), (0, -1), (-1, 0)),
        "default": ((0, 1), (1, 0), (-1, 0), (0, -1)),
    }
    candidates = directions.get(priority, directions["default"])
    for dx, dy in candidates:
        nx = x + dx
        ny = y + dy
        if 0 <= nx < len(grid) and 0 <= ny < len(grid[0]) and grid[nx][ny] == 0:
            yield (nx, ny)
```

这一段的作用

这段代码负责从当前房间出发，找出能继续走到的上下左右邻居房间。

执行逻辑

程序先根据当前优先方向 `priority`，决定邻居的检查顺序。比如优先方向是 `right`，那就先看右，再看下、上、左。然后它逐个尝试这些偏移，计算新坐标 `(nx, ny)`。只有当新坐标没有越界，而且 `grid[nx][ny] == 0` 时，才把这个邻居返回出去。这里的 0 表示可通行，非 0 一般表示障碍。

它是干什么的

这一段既控制“能不能走”，也控制“同样可走时优先往哪边尝试”。所以它不只是列出邻居，还是 A 星路线偏好的一个调味器。

片段 3: A 星的初始化

函数 `a_star()` 的开头

```
def a_star(grid: Sequence[Sequence[int]], start: GridPoint, end: GridPoint, priority:
str = "right") -> Optional[List[GridPoint]]:
    if start == end:
        return [start]

    open_list: List[_Node] = []
    start_node = _Node(f=0, position=start, g=0, h=heuristic(start, end), parent=None)
    start_node.f = start_node.g + start_node.h
    heapq.heappush(open_list, start_node)
    closed = set()
    best_g = {start: 0}
```

这一段的作用

这段代码负责把 A 星搜索的初始状态搭起来。

执行逻辑

如果起点和终点本来就是同一个房间，那直接返回只包含起点的路径。否则，创建开放列表 `open_list`，把起点包装成一个节点压进去。起点的真实代价 `g` 为 0，估计代价 `h` 由曼哈顿距离给出，总分 `f = g + h`。`closed` 集合用于记录已经确认处理过的节点，`best_g` 用来记录某个位置目前找到的最优真实代价，避免反复走更差的路径。

它是干什么的

这一步相当于在地图上摆下第一个探索点，并准备好之后要用的“已处理集合”和“最优代价表”。

片段 4: A 星主循环与回溯路径

函数 `a_star()` 的主体

```
while open_list:
    current = heapq.heappop(open_list)
    if current.position in closed:
        continue
    closed.add(current.position)

    if current.position == end:
        path: List[GridPoint] = []
        node: Optional[_Node] = current
        while node is not None:
            path.append(node.position)
            node = node.parent
        return list(reversed(path))

    for next_pos in _neighbors(current.position, grid, priority):
        next_g = current.g + 1
        if next_g >= best_g.get(next_pos, 10 ** 9):
            continue
        best_g[next_pos] = next_g
        next_h = heuristic(next_pos, end)
        heapq.heappush(
            open_list,
            _Node(
                f=next_g + next_h,
                position=next_pos,
                g=next_g,
                h=next_h,
                parent=current,
            ),
        )
return None
```

这一段的作用

这一段就是 A 星算法真正跑起来的地方。它会持续从开放列表里拿出最优节点，不断向外扩展，直到找到终点。

执行逻辑

程序每次都从堆里弹出 `f` 值最小的节点作为当前节点。如果这个位置之前已经处理过，就跳过。否则把它加入 `closed`。如果当前节点就是终点，那么程序就沿着 `parent` 链一路往回走，把整条路径倒着拼出来，再反转成从起点到终点的顺序。如果不是终点，就去拿它的邻居。对于每个邻居，真实代价 `next_g` 就是当前 `g + 1`。如果这个代价不比之前记录的更优，就不要它；如果更优，就更新 `best_g`，重新计算估价 `h`，再把新节点压入堆。

它是干什么的

这就是整套房间级寻路的核心算法。它不会直接告诉角色在屏幕里怎么走，但会告诉程序从当前房间出发，最短路线应该经过哪些房间。

片段 5：把整条路径压缩成下一步方向

函数 `judge_direction()` 与 `next_direction()`

```
def judge_direction(starting_point: GridPoint, end_point: GridPoint) -> Optional[str]:
    x1, y1 = starting_point
    x2, y2 = end_point
    if x2 > x1:
        return "down"
    if x2 < x1:
        return "up"
    if y2 > y1:
        return "right"
    if y2 < y1:
        return "left"
    return None

def next_direction(grid: Sequence[Sequence[int]], start: GridPoint, end: GridPoint,
priority: str = "right") -> Optional[str]:
    path = a_star(grid, start, end, priority=priority)
    if not path or len(path) < 2:
        return None
    return judge_direction(path[0], path[1])
```

这一段的作用

这段代码把 A 星整条房间路径，进一步压缩成当前最重要的那一个信息：下一步应该往哪边走。

执行逻辑

`next_direction()` 先调用前面的 `a_star()` 算出完整路径。如果路径为空，或者路径长度不足 2，说明当前无法继续前进，就返回空。否则只看路径的前两个房间：当前房间和下一个房间，然后交给 `judge_direction()` 去比较它们的行列变化，从而得出 `up`、`down`、`left`、`right` 之一。

它是干什么的

这是 A 星和整个刷图系统连接的关键出口。动作层不需要整条房间路径，它只需要知道“下一间房在哪个方向”，然后再去当前画面里找对应方向的门。

第五章： `minimap_nav.py`

这个文件负责什么： 这是小地图导航中心。它会裁出小地图区域，用模板匹配识别当前房间、Boss 房、问号房、精英房等，再调用 A 星算出下一步方向，并尝试根据方向选一扇最合适的门。

片段 1：小地图规格与路线快照数据结构

文件顶部数据类和地图配置

```
from copy import deepcopy
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional, Sequence, Tuple

import cv2
import numpy as np

from dnf.door_strategy import DoorCandidate, choose_best_door
from dnf.minimap_astar import next_direction

Point = Tuple[int, int]
Rect = Tuple[int, int, int, int]

@dataclass
class MapSpec:
    name: str
    crop_rect_1067: Rect
    minimap_width: int
    minimap_height: int
    rows: int
    cols: int
    room_grid: List[List[int]]

@dataclass
class RouteSnapshot:
    current_room: Optional[Point]
    boss_room: Optional[Point]
    query_room: Optional[Point]
    elite_room: Optional[Point]
    down_room: Optional[Point]
    next_room_direction: Optional[str]
    selected_door_center: Optional[Tuple[float, float]]
    debug_scores: Optional[Dict[str, float]] = None
```

这一段的作用

这段代码定义了导航器最重要的两个数据结构：地图规格 `MapSpec`，以及路线快照 `RouteSnapshot`。

执行逻辑

`MapSpec` 描述一张地图该如何被理解：小地图裁剪区域在哪、总共几行几列房间、以及房间网格如何表示。`RouteSnapshot` 则是导航器一次计算后的完整输出，其中既包括当前位置和目标房间，也包括下一步方向、选中的门中心，以及调试分数。

它是干什么的

如果说 `detector.py` 负责输出“大画面有什么目标”，那么 `minimap_nav.py` 这里负责输出“副本层级上我现在在哪、该去哪里、下一步往哪走”。

片段 2：生成全可通行网格和地图字典

函数 `_all_walkable()` 与 `MAP_SPECS`

```
def _all_walkable(rows: int, cols: int) -> List[List[int]]:
    return [[0 for _ in range(cols)] for _ in range(rows)]

MAP_SPECS: Dict[str, MapSpec] = {
    "generic": MapSpec(
        name="generic",
        crop_rect_1067=(893, 52, 1055, 142),
        minimap_width=162,
        minimap_height=90,
        rows=5,
        cols=9,
        room_grid=_all_walkable(5, 9),
    ),
    "haibolun": MapSpec(
        name="haibolun",
        crop_rect_1067=(966, 52, 1056, 124),
        minimap_width=90,
        minimap_height=72,
        rows=4,
        cols=5,
        room_grid=_all_walkable(4, 5),
    ),
}
```

这一段的作用

这里定义了不同小地图模板的基本参数，并给每一种地图生成一个“全可通行”的房间网格。

执行逻辑

`_all_walkable()` 会生成一个全 0 的二维数组，其中 0 表示该房间位置可走。然后 `MAP_SPECS` 里列出不同地图名对应的配置，比如 `generic` 和 `haibolun`。每个配置都包含小地图在 1067 宽度画面中的裁剪区域，以及这个地图应该被划分成多少行多少列。

它是干什么的

这一步是在告诉导航器：“不同地图长得不一样，所以你裁图的位置、网格行列数、房间规模都要知道。”当前代码里的房间网格没有真实障碍信息，基本是假设整个网格都可以通行，这也是这份实现的一个局限。

片段 3：初始化导航器并加载模板

类 `MiniMapNavigator` 的 `__init__`

```
class MiniMapNavigator:
    def __init__(self, map_name: str = "generic", assets_dir: Optional[Path] = None,
threshold: float = 0.7):
        if map_name not in MAP_SPECS:
            map_name = "generic"
        self.map_name = map_name
        self.spec = MAP_SPECS[map_name]
        self.threshold = threshold
        self.assets_dir = assets_dir or (Path(__file__).resolve().parent / "res")
        self.templates = {
            "hero": self._load_template("map_hero.png"),
            "boss": self._load_template("map_boss.png"),
            "query": self._load_template("map_query.png"),
            "query1": self._load_template("map_query1.png"),
            "query3": self._load_template("map_query3.png"),
            "query4": self._load_template("map_query4.png"),
            "query5": self._load_template("map_query5.png"),
            "elite": self._load_template("map_elite.png"),
            "special": self._load_template("map_special_room.png"),
        }
        down_template = self._load_template_optional("map_down.png")
        if down_template is not None:
            self.templates["down"] = down_template
        self.last_direction: Optional[str] = None
```

这一段的作用

这段代码负责创建一个可用的小地图导航器，并把所有小图标模板读进内存。

执行逻辑

首先确认传入的地图名称是否存在，如果不存在就退回到 `generic`。然后记录当前使用的地图规格和默认模板匹配阈值。`assets_dir` 默认指向当前文件目录下的 `res` 文件夹。接着把英雄图标、Boss 图标、问号房图标、精英房图标、特殊房图标等全部读进来。`down` 模板是可选的，所以单独用一个“可选加载”函数。最后 `last_direction` 用来记住上一次路线方向，方便后面选门时减少掉头。

它是干什么的

这一步相当于给导航器准备好“识别字典”。之后每次看到小地图，程序都是拿这些模板一张张去比对，判断图标在哪里。

片段 4：加载模板图片

函数 `_load_template()` 与 `_load_template_optional()`

```
def _load_template(self, filename: str) -> np.ndarray:
    path = self.assets_dir / filename
    image = cv2.imread(str(path), cv2.IMREAD_COLOR)
    if image is None:
        raise FileNotFoundError(f"template not found: {path}")
    return image

def _load_template_optional(self, filename: str) -> Optional[np.ndarray]:
    path = self.assets_dir / filename
    if not path.exists():
        return None
    image = cv2.imread(str(path), cv2.IMREAD_COLOR)
    return image
```

这一段的作用

这两段函数负责把小地图模板文件从磁盘读进来。

执行逻辑

`_load_template()` 会严格要求模板必须存在，如果图片没读到，就直接报错，因为这说明主导航逻辑缺少关键模板。`_load_template_optional()` 则更宽松一些，如果文件不存在就返回 `None`，不会中断程序。

它是干什么的

这里区分“必要模板”和“可选模板”，体现的是一种稳定性设计：主流程必须依赖的模板不能丢，可选能力丢了可以退化运行。

片段 5：按分辨率换算小地图裁剪区域

函数 `_scaled_crop_rect()` 与 `extract_minimap()`

```
def _scaled_crop_rect(self, frame: np.ndarray) -> Rect:
    frame_h, frame_w = frame.shape[:2]
    x1, y1, x2, y2 = self.spec.crop_rect_1067
    scale_x = frame_w / 1067.0
    scale_y = frame_h / 600.0
    return (
        int(round(x1 * scale_x)),
        int(round(y1 * scale_y)),
        int(round(x2 * scale_x)),
        int(round(y2 * scale_y)),
    )

def extract_minimap(self, frame: np.ndarray) -> np.ndarray:
    x1, y1, x2, y2 = self._scaled_crop_rect(frame)
    return frame[y1:y2, x1:x2]
```

这一段的作用

这一段负责从整张游戏截图中，把小地图准确裁出来。

执行逻辑

地图配置里存的裁剪区域是基于 1067×600 这类基准尺寸写的，但真实截图不一定总是这个大小。所以程序先拿当前截图的宽高，和基准宽高进行比例换算，计算出当前分辨率下的小地图区域。之后 `extract_minimap()` 根据换算出来的坐标直接把那块图裁剪出来。

它是干什么的

这一步本质上是在做分辨率适配。否则一旦游戏窗口大小略有变化，程序就会裁错位置，小地图识别就全乱了。

片段 6：模板匹配

函数 `_match_template()` 与 `_match_template_debug()`

```
def _match_template(self, image: np.ndarray, template: np.ndarray, threshold:
Optional[float] = None) -> List[Tuple[float, float]]:
    threshold = self.threshold if threshold is None else threshold
    image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    template_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
    result = cv2.matchTemplate(image_gray, template_gray, cv2.TM_CCOEFF_NORMED)
    ys, xs = np.where(result >= threshold)
    matches: List[Tuple[float, float]] = []
    h, w = template_gray.shape[:2]
    for y, x in zip(ys, xs):
        matches.append((x + w / 2.0, y + h / 2.0))
    return matches

def _match_template_debug(self, image: np.ndarray, template: np.ndarray) ->
Tuple[List[Tuple[float, float]], float]:
    image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    template_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
    result = cv2.matchTemplate(image_gray, template_gray, cv2.TM_CCOEFF_NORMED)
    _, max_val, _, _ = cv2.minMaxLoc(result)
    ys, xs = np.where(result >= self.threshold)
    matches: List[Tuple[float, float]] = []
    h, w = template_gray.shape[:2]
    for y, x in zip(ys, xs):
        matches.append((x + w / 2.0, y + h / 2.0))
    return matches, float(max_val)
```

这一段的作用

这是小地图识别的核心：模板匹配。程序拿一张小图标，在小地图上滑动比较，看哪里最像它。

执行逻辑

首先把大图和模板都转成灰度图，减少颜色差异带来的影响。然后调用

`cv2.matchTemplate(..., TM_CCOEFF_NORMED)` 做归一化相关性匹配，得到一张相似度热力图。程序会找出所有相似度大于阈值的位置，再把这些位置换算成模板中心点。调试版的函数则额外取出全局最大相似度 `max_val`，方便你判断某个模板当前到底匹配得好不好。

它是干什么的

这就是导航器的“第二只眼睛”。它不识别大画面里的怪和门，而是专门识别 UI 级别的小图标，比如玩家当前位置图标、Boss 图标、问号房图标。

片段 7：把像素位置换算成房间坐标

函数 `compute_room_id()`

```
def compute_room_id(self, x: float, y: float, minimap_image: np.ndarray) -> Point:
    height, width = minimap_image.shape[:2]
    cell_w = width / self.spec.cols
    cell_h = height / self.spec.rows
    row = int(y // cell_h)
    col = int(x // cell_w)
    row = max(0, min(self.spec.rows - 1, row))
    col = max(0, min(self.spec.cols - 1, col))
    return row, col
```

这一段的作用

前面模板匹配只知道“图标在小地图里的哪个像素位置”，这一段负责把这个像素位置转换成“第几行第几列的房间”。

执行逻辑

程序先算出小地图每一个房间格子的宽和高。然后用图标中心点除以格子尺寸，得到它落在哪一行哪一列。最后再用 `max` 和 `min` 把结果夹住，防止因为边缘误差越界。

它是干什么的

这是把“图像空间”转换成“副本网格空间”的关键一步。A 星吃的不是像素坐标，而是房间坐标，所以必须经过这一层换算。

片段 8：识别当前房间和目标房间标记

函数 `detect_room_markers()`

```
def detect_room_markers(self, frame: np.ndarray) -> Dict[str, Optional[Point]]:
    minimap = self.extract_minimap(frame)
    current_room = None
    boss_room = None
    query_room = None
    elite_room = None
    down_room = None

    hero_matches = self._match_template(minimap, self.templates["hero"])
    if hero_matches:
        current_room = self.compute_room_id(*hero_matches[0], minimap)

    boss_matches = self._match_template(minimap, self.templates["boss"])
    if boss_matches:
        boss_room = self.compute_room_id(*boss_matches[0], minimap)

    query_matches = self._match_template(minimap, self.templates["query"])
    if not query_matches:
        query_matches = self._match_template(minimap, self.templates["query1"])
    if not query_matches:
        query_matches = self._match_template(minimap, self.templates["query3"])
    if not query_matches:
        query_matches = self._match_template(minimap, self.templates["query4"])
    if not query_matches:
        query_matches = self._match_template(minimap, self.templates["query5"])
    if query_matches:
        query_room = self.compute_room_id(*query_matches[0], minimap)

    elite_matches = self._match_template(minimap, self.templates["elite"])
    if elite_matches:
        elite_room = self.compute_room_id(*elite_matches[0], minimap)

    special_matches = self._match_template(minimap, self.templates["special"])
    if special_matches and elite_room is None:
        elite_room = self.compute_room_id(*special_matches[0], minimap)

    down_template = self.templates.get("down")
    if down_template is not None:
        down_matches = self._match_template(minimap, down_template,
threshold=0.62)
        if down_matches:
            down_room = self.compute_room_id(*down_matches[0], minimap)

    return {
        "current_room": current_room,
        "boss_room": boss_room,
        "query_room": query_room,
        "elite_room": elite_room,
```

```
"down_room": down_room,  
}
```

这一段的作用

这段代码把整张小地图上的重要房间全部识别出来，包括当前房间、Boss 房、问号房、精英房和下层入口房。

执行逻辑

首先从整张截图里裁出小地图。然后逐类模板去匹配。英雄图标用来确定当前房间，Boss 图标对应 Boss 房。问号房比较特殊，代码准备了多种不同样式的模板，只要匹配到其中一种就认作问号房。精英房先尝试常规模板，如果没有匹配到，就用特殊房模板兜底。下层入口则采用一个单独较低一点的阈值 `0.62` 去识别，因为这类图标可能更难匹配。最后把所有识别结果作为一个字典返回。

它是干什么的

这一步是在告诉程序：“我现在身处哪间房，值得去的目标房分别在哪里。”后面目标房的选择和 A 星的起终点，都靠这一段提供信息。

片段 9：调试分数与门候选收集

函数 `get_debug_scores()` 与 `_door_candidates_from_objects()`

```
def get_debug_scores(self, frame: np.ndarray) -> Dict[str, float]:
    minimap = self.extract_minimap(frame)
    scores: Dict[str, float] = {}
    for name, template in self.templates.items():
        _, max_val = self._match_template_debug(minimap, template)
        scores[name] = round(max_val, 4)
    return scores

def _door_candidates_from_objects(self, objects: Sequence[dict]) ->
List[DoorCandidate]:
    doors: List[DoorCandidate] = []
    for item in objects:
        if "door" not in item:
            continue
        xywh = item["door"]["xywh"]
        if len(xywh) != 4:
            continue
        x, y, w, h = xywh
        if w <= 0 or h <= 0:
            continue
        center = (x + w / 2.0, y + h / 2.0)
        doors.append(DoorCandidate(bbox=xywh, center=center))
    return doors
```

这一段的作用

这一段一半是给开发者看的调试工具，一半是给门策略做准备的数据转换。

执行逻辑

`get_debug_scores()` 会对每个模板跑一遍调试版匹配函数，并记录最大匹配分值。这样开发时可以直接看到，是不是某种模板现在分数太低，需要换图或调阈值。

`_door_candidates_from_objects()` 则遍历 YOLO 检测结果，把所有门对象挑出来，转换成统一的 `DoorCandidate` 列表，顺带过滤掉格式异常或宽高无效的框。

它是干什么的

前者帮助你排查“小地图为什么没识别出来”，后者帮助你把“大画面里检测出的门”交给门评分算法。这两件事共同服务于导航器的最终决策。

片段 10：决定本轮要去哪个目标房

函数 `_pick_target_room()`

```
def _pick_target_room(
    self,
    current_room: Optional[Point],
    query_room: Optional[Point],
    down_room: Optional[Point],
    elite_room: Optional[Point],
    boss_room: Optional[Point],
    prefer_special_room: bool,
) -> Optional[Point]:
    candidates: List[Optional[Point]] = []
    if prefer_special_room:
        candidates.extend([query_room, down_room, elite_room, boss_room])
    else:
        candidates.append(boss_room)

    for room in candidates:
        if room is None:
            continue
        if current_room is not None and room == current_room:
            continue
        return room
    return None
```

这一段的作用

这段代码负责在多个候选目标房中，选出当前这一轮最值得前往的那个房间。

执行逻辑

如果 `prefer_special_room` 为真，候选顺序就是问号房、下层房、精英房、Boss 房。也就是说，它优先去特殊房，再去 Boss。若不偏好特殊房，就只考虑 Boss 房。之后它按顺序遍历候选列表，跳过空值和当前所在房间，只要找到第一个可用目标，就直接返回。

它是干什么的

这其实是副本路线策略的体现。A 星只会算“从哪到哪怎么走”，但“终点到底选哪间房”这件事，是这段目标房选择逻辑决定的。

片段 11：生成路线快照并调用 A 星

函数 `build_route_snapshot()` 的前半段

```
def build_route_snapshot(
    self,
    frame_bgr: np.ndarray,
    detection_objects: Sequence[dict],
    prefer_special_room: bool = True,
) -> RouteSnapshot:
    room_info = self.detect_room_markers(frame_bgr)
    current_room = room_info["current_room"]
    boss_room = room_info["boss_room"]
    query_room = room_info["query_room"]
    elite_room = room_info["elite_room"]
    down_room = room_info["down_room"]

    target_room = self._pick_target_room(
        current_room=current_room,
        query_room=query_room,
        down_room=down_room,
        elite_room=elite_room,
        boss_room=boss_room,
        prefer_special_room=prefer_special_room,
    )

    next_room_direction = None
    if current_room is not None and target_room is not None:
        next_room_direction = next_direction(deepcopy(self.spec.room_grid),
        current_room, target_room, priority="right")
```

这一段的作用

这段代码开始把前面所有小地图识别结果整合起来，真正得出“下一间房应该往哪边走”。

执行逻辑

它先调用 `detect_room_markers()` 获取当前房间、Boss 房、问号房等位置。然后调用 `_pick_target_room()` 选定目标房。接下来，只要当前房间和目标房都存在，就调用 `next_direction()`，把当前房间和目标房丢给 A 星，让它返回下一步方向。这里用 `deepcopy(self.spec.room_grid)` 是为了防止后续算法在网格上做改动时影响原始配置。

它是干什么的

这段就是“小地图识别”和“A 星寻路”的结合点。也正是在这里，程序第一次把视觉信息转成了一个真正可执行的方向提示。

片段 12：根据方向尝试选门并产出完整路线快照

函数 `build_route_snapshot()` 的后半段

```
selected_door_center = None
if next_room_direction:
    player_center = None
    for item in detection_objects:
        if "player" in item:
            x, y, w, h = item["player"]["xywh"]
            player_center = (x + w / 2.0, y + h / 2.0)
            break
    if player_center is not None:
        selected_door = choose_best_door(
            self._door_candidates_from_objects(detection_objects),
            player_center=player_center,
            expected_direction=next_room_direction,
            last_direction=self.last_direction,
        )
        if selected_door is not None:
            selected_door_center = selected_door.center
            self.last_direction = next_room_direction

debug_scores = self.get_debug_scores(frame_bgr)
return RouteSnapshot(
    current_room=current_room,
    boss_room=boss_room,
    query_room=query_room,
    elite_room=elite_room,
    down_room=down_room,
    next_room_direction=next_room_direction,
    selected_door_center=selected_door_center,
    debug_scores=debug_scores,
)
```

这一段的作用

这一段在 A 星得到房间方向之后，继续把这个方向和大画面里的门关联起来，尝试找出当前画面中最合适的那扇门。

执行逻辑

如果 A 星已经给出了方向，那程序就先从检测结果里找玩家的中心点。找到后，再把所有门整理成 `DoorCandidate` 列表，然后调用 `choose_best_door()`。如果选门成功，就把门的中心点记录到 `selected_door_center`，并把当前方向保存成 `last_direction`。最后再补上一份模板匹配调试分数，把所有信息打包成一个 `RouteSnapshot` 返回。

它是干什么的

这一步说明：A 星并不是孤立存在的。它的方向结果会进一步影响当前房间的门选择。虽然当前主流程最后没有直接把这个门坐标传给 `Game` 使用，但这里已经完成了“房间方向 -> 门方向”的一次关联。

第六章： `game.py`

这个文件负责什么： 这是整套刷图行为的控制中心。它决定这一帧到底应该打怪、捡东西、朝哪扇门移动、何时判断自己卡住、以及卡住后怎么恢复。你可以把它理解成“手和脚”。

片段 1: 类级状态与初始化参数

类 `Game` 的开头

```
class Game:
    _action_cache: Optional[str] = None
    _player: Optional[BoxDict] = None
    _pre_player: Optional[BoxDict] = None
    _index = 0
    _player_missing_count = 0
    _position_history: List[Tuple[float, float]] = []
    _stuck_count = 0
    _last_recover_time = 0.0
    _recover_step = 0
    _last_player_center: Optional[Tuple[float, float]] = None
    _room_entry_position: Optional[Tuple[float, float]] = None
    _room_entry_time = 0.0
    _flow_direction: Optional[str] = None
    _missing_player_recover_threshold = 10
    _route_search_direction: Optional[str] = None
    _route_search_phase = 0
    _route_search_start: Optional[Tuple[float, float]] = None

    def __init__(self, obj: Sequence[dict], width: int, height: int, direction):
        self._obj = obj or []
        self._width = width
        self._height = height
        self._player_xywh: Optional[List[float]] = None
        self._attack_x = 100
        self._attack_y = 100
        self._move_x = 20
        self._move_y = 20
        self._direction = direction
```

这一段的作用

这段代码先把动作层需要的所有“记忆变量”和“阈值参数”准备出来。

执行逻辑

类变量部分用于跨帧记忆状态，比如上一帧按着哪个方向、上一帧玩家位置、最近几帧位置历史、进入房间的时间、卡住恢复走到第几步等。实例初始化部分则接收本轮检测结果、画面宽高和路线方向，并设置攻击范围和拾取范围阈值。

它是干什么的

这一段相当于动作系统的“记忆仓库”。因为刷图不是一帧就结束，而是连续很多帧组成的行为，所以必须有跨帧状态，程序才知道自己是不是卡住了、刚从哪扇门进来、上一轮是不是在朝某个方向移动。

片段 2：读取方向提示与取检测对象

工具函数 `_current_door_direction()`、`_get_cls()`、`_get_cls()`

```
def _current_door_direction(self) -> Optional[str]:
    if isinstance(self._direction, (list, tuple)):
        if not self._direction:
            return None
        if Game._index >= len(self._direction):
            return str(self._direction[-1]).upper()
        return str(self._direction[Game._index]).upper()
    if isinstance(self._direction, str) and self._direction.strip():
        return self._direction.strip().upper()
    return None

def _get_cls(self, cls_name: str) -> Optional[BoxDict]:
    for item in self._obj:
        if cls_name in item:
            return item[cls_name]
    return None

def _get_cls(self, cls_name: str) -> List[BoxDict]:
    result = []
    for item in self._obj:
        if cls_name in item:
            result.append(item[cls_name])
    return result
```

这一段的作用

这几个函数是动作层最常用的工具函数。一个负责从导航结果里拿方向提示，另外两个负责从 YOLO 检测列表里拿目标对象。

执行逻辑

`_current_door_direction()` 兼容两种输入：如果传进来的方向是一个列表，它会根据 `Game._index` 取当前应走的那一步；如果是一个字符串，就直接返回大写版本。

`_get_cls()` 只取第一个匹配某类的对象，而 `_get_cls()` 会把所有同类对象都收集起来，例如所有怪物、所有金币、所有门。

它是干什么的

本质上是在给动作层提供统一的数据读取接口，让后面的战斗、拾取和过门逻辑都能用同一套方式去拿目标。

片段 3：释放按键与距离计算

工具函数 `_release_direction_keys()`、`_release_cached_action()`、`_distance()`

```
def _release_direction_keys(self) -> None:
    for key in ("up", "down", "left", "right"):
        pydirectinput.keyUp(key)

def _release_cached_action(self) -> None:
    if not Game._action_cache:
        self._release_direction_keys()
        return
    for action in Game._action_cache.strip().split("_"):
        pydirectinput.keyUp(action.lower())
    self._release_direction_keys()
    Game._action_cache = None

def _distance(self, a: Tuple[float, float], b: Tuple[float, float]) -> float:
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5
```

这一段的作用

这段代码负责两个很基础但非常重要的能力：第一，安全地松开当前按着的方向键；第二，计算两个点之间的直线距离。

执行逻辑

`_release_direction_keys()` 会把上下左右四个方向键都松开。

`_release_cached_action()` 更细一些，它先根据缓存动作字符串，比如 `RIGHT_UP`，把对应的键全部抬起，然后再调用一次总释放，最后把缓存动作清空。`_distance()` 则返回标准欧几里得距离，后面判断“门是否靠近入口”“角色是否突然跳到另一个位置”时都要用到。

它是干什么的

这类工具函数虽然不显眼，但非常关键。自动化程序最怕“键按住不放”或“前一帧的动作影响下一帧”，所以释放按键逻辑必须足够稳。

片段 4：推断进房方向与更新房间入口信息

函数 `_infer_flow_direction_from_spawn()` 与 `_maybe_update_room_entry()`

```
def _infer_flow_direction_from_spawn(self, player_center: Tuple[float, float]) -> Optional[str]:
    x, y = player_center
    edge_scores = {
        "LEFT": x,
        "RIGHT": self._width - x,
        "UP": y,
        "DOWN": self._height - y,
    }
    nearest_edge = min(edge_scores, key=edge_scores.get)
    opposite = {
        "LEFT": "RIGHT",
        "RIGHT": "LEFT",
        "UP": "DOWN",
        "DOWN": "UP",
    }
    if edge_scores[nearest_edge] > min(self._width, self._height) * 0.28:
        return None
    return opposite[nearest_edge]

def _maybe_update_room_entry(self) -> None:
    if not self._player_xywh:
        return
    player_center = (self._player_xywh[0], self._player_xywh[1])
    previous = Game._last_player_center
    Game._last_player_center = player_center
    if previous is None:
        Game._room_entry_position = player_center
        Game._room_entry_time = time.time()
        Game._flow_direction =
self._infer_flow_direction_from_spawn(player_center)
        return

    jump_distance = self._distance(player_center, previous)
    if jump_distance >= 150:
        Game._room_entry_position = player_center
        Game._room_entry_time = time.time()
        Game._flow_direction =
self._infer_flow_direction_from_spawn(player_center)
        logger.info(
            "room transition inferred, entry={}, flow_direction={}",
            Game._room_entry_position,
            Game._flow_direction,
        )
```

这一段的作用

这段代码的任务是推断角色刚进入一个新房间时，是从哪个边缘出现的，从而反推出副本推进的大致流向。

执行逻辑

先看角色离屏幕四个边缘哪个最近。如果角色离左边最近，那通常意味着他是从左侧门进来的，因此接下来的推进方向大概率是往右。代码用一个 `0.28` 的比例阈值判断角色是否真的靠边，不靠边就不强行推断。`_maybe_update_room_entry()` 则会在每一帧更新玩家中心点，并对比上一帧位置。如果第一次记录玩家位置，就直接把当前位置记成入口；如果和上一帧距离突然跳了 150 像素以上，就认为可能刚刚换房，于是更新入口点、更新时间和推断出来的流向。

它是干什么的

这一步很重要，因为后面为了防止角色刚进房就回头走，会用到“入口保护”和“反向门过滤”。它的核心思想是：程序要记住自己是从哪里进来的。

片段 5: 过滤回头门

函数 `_filter_backtrack_doors()`、`_is_entry_protected()`、`_should_block_backtrack_direction()`

```

def _filter_backtrack_doors(self, doors: Sequence[BoxDict], direction_hint:
Optional[str]) -> List[BoxDict]:
    if not doors:
        return []

    if not Game._room_entry_position:
        return list(doors)

    now = time.time()
    protect_entry = now - Game._room_entry_time <= 3.0
    if not protect_entry:
        return list(doors)

    entry = Game._room_entry_position
    entry_side = self._reverse_direction(Game._flow_direction)
    filtered: List[BoxDict] = []
    for door in doors:
        x, y, w, h = door["xywh"]
        center = (x + w / 2.0, y + h / 2.0)
        near_entry = self._distance(center, entry) <= 240
        dx = center[0] - entry[0]
        dy = center[1] - entry[1]
        if abs(dx) >= abs(dy):
            door_side = "RIGHT" if dx > 0 else "LEFT"
        else:
            door_side = "DOWN" if dy > 0 else "UP"

        if not near_entry:
            filtered.append(door)
            continue

        if entry_side and door_side == entry_side:
            logger.info("ignore backtrack door at entry side {}: {}".format(
center)
                entry_side,
                continue

        reverse_hint = self._reverse_direction(direction_hint)
        if reverse_hint and door_side == reverse_hint:
            logger.info("ignore reverse door for hint {}: {}".format(
center)
                direction_hint,
                continue

        filtered.append(door)

    return filtered

def _is_entry_protected(self) -> bool:
    return bool(Game._room_entry_position) and (time.time() -
Game._room_entry_time <= 3.0)

```

```
def _should_block_backtrack_direction(self, direction: Optional[str]) -> bool:
    if not direction or not self._is_entry_protected():
        return False
    entry_side = self._reverse_direction(Game._flow_direction)
    if not entry_side:
        return False
    return direction.upper() == entry_side
```

这一段的作用

这一段专门解决一个很常见的问题：角色刚进房，还没清怪，就因为门检测或路线判断抖动，立刻又跑回上一间房。

执行逻辑

过滤逻辑只在进房后的 3 秒保护期内生效。程序会拿每扇门和“入口位置”比较，先判断这扇门是不是离入口很近。只有离入口很近的门才会被重点检查。如果这扇门刚好处在“入口反方向的回头边”，就直接忽略；如果这扇门虽然不一定是入口门，但它正好与当前路线提示方向相反，也会被忽略。后面的 `_should_block_backtrack_direction()` 则是在没有明确门可选时，连后备方向都要做最后一道防回头检查。

它是干什么的

这是一套“防抖”和“防走回头路”的启发式规则。它不复杂，但对刷图稳定性特别重要，因为很多自动化程序就是死在频繁掉头和原地来回穿门上。

片段 6：基础按键动作与方向反转

函数 `_key_press()`、`_tap_direction()`、`_reverse_direction()`

```
def _key_press(self, key: str) -> None:
    pydirectinput.keyDown(key)
    time.sleep(0.08)
    pydirectinput.keyUp(key)
    time.sleep(0.05)

def _tap_direction(self, direction: str, duration: float = 0.12) -> None:
    for action in direction.strip().split("_"):
        pydirectinput.keyDown(action.lower())
    time.sleep(duration)
    for action in direction.strip().split("_"):
        pydirectinput.keyUp(action.lower())
    time.sleep(0.04)

def _reverse_direction(self, direction: Optional[str]) -> Optional[str]:
    mapping = {
        "LEFT": "RIGHT",
        "RIGHT": "LEFT",
        "UP": "DOWN",
        "DOWN": "UP",
        "LEFT_UP": "RIGHT_DOWN",
        "LEFT_DOWN": "RIGHT_UP",
        "RIGHT_UP": "LEFT_DOWN",
        "RIGHT_DOWN": "LEFT_UP",
    }
    return mapping.get(direction or "")
```

这一段的作用

这里封装了动作层最基础的操作能力：按一下某个键、轻点某个方向、以及获取某个方向的反方向。

执行逻辑

`_key_press()` 适合攻击、拾取这类短按操作，按下后等 0.08 秒再松开。

`_tap_direction()` 可以处理单方向和斜方向，比如 `RIGHT_UP` 会拆成同时按右和上，持续一小段时间后松开。`_reverse_direction()` 则提供一张方向映射表，后面卡住恢复和回头过滤都会大量使用它。

它是干什么的

如果把整套程序看成人玩游戏，这里就是“手指动作库”。上层只要说“按 x”“轻点右上”，底层就由这些函数具体发出键盘事件。

片段 7: 选最近目标

函数 `_get_nearest()`

```
def _get_nearest(self, cls_objs: Sequence[BoxDict], direct: Optional[str] = None)
-> Optional[BoxDict]:
    nearest = None
    min_distance = float("inf")

    if not self._player_xywh:
        return None

    for item in cls_objs:
        xywh = list(item["xywh"])
        obj_x = xywh[0] + xywh[2] / 2
        obj_y = xywh[1] + xywh[3] / 2
        player_x = self._player_xywh[0]
        player_y = self._player_xywh[1]
        dx = obj_x - player_x
        dy = obj_y - player_y

        if direct:
            name = direct.lower()
            if name == "right" and dx <= 0:
                continue
            if name == "left" and dx >= 0:
                continue
            if name == "up" and dy >= 0:
                continue
            if name == "down" and dy <= 0:
                continue

        distance = (dx ** 2 + dy ** 2) ** 0.5
        if distance < min_distance:
            min_distance = distance
            nearest = {
                "xywh": [obj_x, obj_y, xywh[2], xywh[3]],
                "conf": item.get("conf", 0),
            }

    return nearest
```

这一段的作用

这一段负责在一堆同类目标里，比如一堆怪物或一堆门里，找出离玩家最近的那个。

执行逻辑

程序先确保自己知道玩家位置。然后对每个目标，把原始框换算成中心点，与玩家中心做差得到 `dx`、`dy`。如果传进来了方向限制 `direct`，就只保留该方向上的目标，比如期望往右走时只保留 `dx > 0` 的目标。之后用欧氏距离算当前目标离玩家多远，只要比目前最小距离还小，就更新为新的最近目标。

它是干什么的

战斗、拾取和过门都大量依赖这个函数。它体现的是一个很朴素但实用的策略：优先处理离自己最近、最容易立刻交互到的目标。

片段 8：把目标相对位置翻译成方向字符串

函数 `_get_direction()`

```
def _get_direction(self, obj_box: Sequence[float]) -> Optional[str]:
    if not self._player_xywh:
        return None

    dx = obj_box[0] - self._player_xywh[0]
    dy = obj_box[1] - self._player_xywh[1]

    if dy < 0:
        if abs(dy) < 20:
            return "RIGHT" if dx > 0 else "LEFT"
        if self._player_xywh[1] - obj_box[1] < abs(dx):
            return "RIGHT_UP" if dx > 0 else "LEFT_UP"
        return "UP"
    if dy > 0:
        if abs(dy) < 20:
            return "RIGHT" if dx > 0 else "LEFT"
        if dy < abs(dx):
            return "RIGHT_DOWN" if dx > 0 else "LEFT_DOWN"
        return "DOWN"
    return None
```

这一段的作用

这段代码负责把“目标在玩家的哪边”翻译成程序能执行的方向字符串，比如 `LEFT`、`UP`、`RIGHT_DOWN`。

执行逻辑

程序先计算目标中心相对玩家中心的横纵偏移。如果目标在玩家上方，先看上下差是否很小，若很小就当成左右方向；否则再比较上下偏差和左右偏差，决定是“偏斜上方”还是“纯上方”。目标在下方时也是同样逻辑。这里的 `20` 像素阈值就是一种容差，允许目标虽然略高一点点，但仍然被当作左右方向处理。

它是干什么的

这是动作层把空间关系变成按键方向的翻译器。后面的打怪、拾取、朝门移动，几乎都是先调用它，再把结果扔进按键函数。

片段 9：真正的移动执行器

函数 `_move()`

```
def _move(
    self,
    direction: str,
    is_slow: bool = False,
    _action_cache: Optional[str] = None,
    press_time: float = 0.1,
    release_time: float = 0.1,
) -> str:
    logger.info("current direction: {}, cached direction: {}", direction,
        _action_cache)
    if _action_cache and direction != _action_cache:
        for action in _action_cache.strip().split("_"):
            pydirectinput.keyUp(action.lower())

    for action in direction.strip().split("_"):
        pydirectinput.keyDown(action.lower())

    if not is_slow:
        time.sleep(press_time)
        for action in direction.strip().split("_"):
            pydirectinput.keyUp(action.lower())
        time.sleep(release_time)
        for action in direction.strip().split("_"):
            pydirectinput.keyDown(action.lower())

    return direction
```

这一段的作用

这就是动作层真正把“方向字符串”转成按键事件的核心函数。

执行逻辑

如果上一帧缓存的方向和当前方向不同，就先把旧方向涉及的键松开，然后按下新方向对应的键。如果不是慢速模式，还会执行一次“按住一小段时间 -> 松开 -> 再按下”的节奏，这样更像连续移动。最后返回当前方向，供外面更新到 `_action_cache` 里。

它是干什么的

你可以把它理解成动作控制层的“总马达”。所有“朝怪靠近”“朝门走”“后备路线移动”等操作，最后都会落到这个函数上。

片段 10：判断角色是否卡住

函数 `_update_motion_state()`

```
def _update_motion_state(self, expect_motion: bool, player_visible: bool) -> bool:
    if not self._player_xywh:
        return False
    if not expect_motion or not player_visible:
        Game._position_history.clear()
        Game._stuck_count = 0
        return False

    current = (self._player_xywh[0], self._player_xywh[1])
    Game._position_history.append(current)
    if len(Game._position_history) > 8:
        Game._position_history.pop(0)

    if len(Game._position_history) < 6:
        return False

    xs = [point[0] for point in Game._position_history]
    ys = [point[1] for point in Game._position_history]
    spread_x = max(xs) - min(xs)
    spread_y = max(ys) - min(ys)
    if spread_x < 18 and spread_y < 18:
        Game._stuck_count += 1
    else:
        Game._stuck_count = 0

    return Game._stuck_count >= 2
```

这一段的作用

这段代码专门判断：“程序明明在尝试移动，但角色是不是其实几乎没动？”如果是，就把它视为卡住。

执行逻辑

只有在两个条件都满足时才会做卡住检测：第一，程序当前确实在期待角色运动；第二，玩家目标本身确实被检测到了。然后它会记录最近几帧玩家中心点位置，最多保留 8 个。如果历史点不够 6 个，先不下结论。等样本够了以后，程序计算这些点在 X 和 Y 方向的扩散范围。若横向和纵向扩散都小于 18 像素，就认为角色基本没动，卡住计数加 1；否则就清零。连续两次满足这个条件，才正式判断为卡住。

它是干什么的

这是一套很典型的“位置历史容差法”。它不要求程序精准知道碰撞发生了什么，只看结果：最近一段时间位置是不是几乎没变。

片段 11：卡住恢复与丢失玩家恢复

函数 `_recover_from_stuck()` 与 `_recover_missing_player()`

```
def _recover_from_stuck(self, reason: str) -> None:
    now = time.time()
    if now - Game._last_recover_time < 1.2:
        return

    try:
        Game._last_recover_time = now
        Game._stuck_count = 0
        Game._position_history.clear()
        logger.warning("stuck recovery triggered: {}", reason)
        current_hint = Game._action_cache or self._current_door_direction() or
Game._flow_direction or "RIGHT"
        self._release_cached_action()
        reverse_hint = self._reverse_direction(current_hint) or "LEFT"
        pattern = Game._recover_step % 4
        Game._recover_step += 1

        if pattern == 0:
            self._tap_direction(reverse_hint, 0.18)
            self._tap_direction("DOWN", 0.12)
        elif pattern == 1:
            self._tap_direction(reverse_hint, 0.18)
            self._tap_direction("UP", 0.12)
        elif pattern == 2:
            for step in ("RIGHT", "DOWN", "LEFT", "UP"):
                self._tap_direction(step, 0.1)
        else:
            self._tap_direction(reverse_hint, 0.22)
            self._tap_direction(current_hint if isinstance(current_hint, str) else
"RIGHT", 0.1)

        Game._action_cache = None
    except Exception as exc:
        logger.exception("stuck recovery failed: {}", exc)
        self._release_cached_action()
        Game._action_cache = None

def _recover_missing_player(self) -> None:
    if Game._player_missing_count < Game._missing_player_recover_threshold:
        return

    logger.warning("player lost for {} frames, try recovery",
Game._player_missing_count)
    self._recover_from_stuck("player_missing")
    Game._player_missing_count = 0
```

这一段的作用

一旦程序判断自己卡住，或者连续很多帧都找不到玩家，它就会进入恢复逻辑，尝试把角色重新从僵局里拽出来。

执行逻辑

首先有一个恢复冷却，避免短时间反复乱按。然后清空卡住计数和位置历史，并选择一个“当前提示方向”。这个提示方向的来源优先级是：当前缓存动作，其次是路线方向，再其次是房间流向，实在不行默认往右。然后计算反方向，按四种模式轮流尝试脱困：反方向再下、反方向再上、绕一个小矩形、反方向再朝当前方向试探。`_recover_missing_player()` 则在连续很多帧都丢失玩家检测结果时，直接复用这套卡住恢复逻辑。

它是干什么的

这是动作层的“自救系统”。因为自动化里总会遇到碰撞、门槛、怪物挡路、识别抖动等问题，如果没有恢复策略，程序很容易永远卡在某个角落。

片段 12: 打怪逻辑

函数 `_kill_monster()`

```
def _kill_monster(self, obj_box: Sequence[float]) -> None:
    if not self._player_xywh:
        return

    if abs(obj_box[0] - self._player_xywh[0]) < self._attack_x and abs(obj_box[1]
- self._player_xywh[1]) < self._attack_y:
        direction = self._get_direction(obj_box)
        face = None
        if direction:
            for item in direction.split("_"):
                if "RIGHT" in item:
                    face = "RIGHT"
                elif "LEFT" in item:
                    face = "LEFT"
                break

        if face:
            self._key_press(face.lower())
            self._key_press("x")
            self._release_cached_action()
            return

        direction = self._get_direction(obj_box)
        if direction:
            Game._action_cache = self._move(direction, is_slow=True,
_action_cache=Game._action_cache)
```

这一段的作用

这段代码负责处理“我要去打这个怪”的动作。逻辑非常直接：够近就打，不够近就靠过去。

执行逻辑

程序先看怪物中心和玩家中心在 X、Y 方向上的差值，是否都进入了攻击范围阈值

`_attack_x` 和 `_attack_y`。如果进入范围，就先根据方向决定角色朝左还是朝右转身，然后按一下 `x` 进行攻击，再把之前缓存的移动动作释放掉。如果还没进入攻击范围，就计算怪物方向，调用 `_move(..., is_slow=True)` 慢慢朝怪物靠近。

它是干什么的

这说明当前战斗不是复杂技能连招，而是一套“接近目标 -> 面向目标 -> 按攻击键”的基础打法。对小白来说，这正好很容易读懂：程序其实就是在模仿最简单的手动打怪流程。

片段 13: 拾取逻辑与朝门移动

函数 `_pick_up()` 与 `_move_to_door()`

```
def _pick_up(self, obj_box: Sequence[float]) -> None:
    if not self._player_xywh:
        return

    if abs(obj_box[0] - self._player_xywh[0]) < self._move_x and abs(obj_box[1] -
self._player_xywh[1]) < self._move_y:
        self._key_press("f")
        self._release_cached_action()
        return

    direction = self._get_direction(obj_box)
    if direction:
        Game._action_cache = self._move(direction, is_slow=True,
_action_cache=Game._action_cache)

def _move_to_door(self, obj_box: Sequence[float]) -> None:
    direction = self._get_direction(obj_box)
    if direction:
        Game._action_cache = self._move(direction,
_action_cache=Game._action_cache)
```

这一段的作用

这段代码分别负责两件事：捡掉落物，以及朝某一扇门移动。

执行逻辑

`_pick_up()` 的逻辑和打怪很像：如果已经进入拾取范围，就按 `f`，并释放掉之前的移动动作；否则就朝物品方向慢慢靠近。`_move_to_door()` 更简单，它先计算门的方向，然后调用 `_move()` 朝门走过去。

它是干什么的

这两个函数说明当前系统里的行为控制其实是一致的：对怪、对物品、对门，本质都是“先计算相对方向，再决定是交互还是继续靠近”。

片段 14：向上路线的特殊搜索模式

函数 `_reset_route_search()` 与 `_fallback_up_search_move()`

```
def _reset_route_search(self) -> None:
    Game._route_search_direction = None
    Game._route_search_phase = 0
    Game._route_search_start = None

def _fallback_up_search_move(self) -> None:
    if not self._player_xywh:
        Game._action_cache = self._move("UP", is_slow=True,
        _action_cache=Game._action_cache)
        return

    current = (self._player_xywh[0], self._player_xywh[1])
    if Game._route_search_direction != "UP":
        Game._route_search_direction = "UP"
        Game._route_search_phase = 0
        Game._route_search_start = current

    if Game._route_search_start is None:
        Game._route_search_start = current

    start_x, start_y = Game._route_search_start
    current_x, current_y = current
    phase = Game._route_search_phase
    amplitude = 20.0

    if phase == 0:
        command = "DOWN"
        if current_y - start_y >= amplitude:
            Game._route_search_phase = 1
            Game._route_search_start = current
            command = "RIGHT"
    elif phase == 1:
        command = "RIGHT"
        if current_x - start_x >= amplitude:
            Game._route_search_phase = 2
            Game._route_search_start = current
            command = "UP"
    elif phase == 2:
        command = "UP"
        if start_y - current_y >= amplitude:
            Game._route_search_phase = 3
            Game._route_search_start = current
            command = "RIGHT"
    else:
        command = "RIGHT"
        if current_x - start_x >= amplitude:
            Game._route_search_phase = 0
            Game._route_search_start = current
            command = "DOWN"
```

```
logger.info(  
    "up fallback sweep: phase={}, start={}, current={}, command={}",  
    Game._route_search_phase,  
    Game._route_search_start,  
    current,  
    command,  
)  
Game._action_cache = self._move(command, is_slow=True,  
_action_cache=Game._action_cache)
```

这一段的作用

这段代码是一个很有意思的特例：当后备路线提示要求向上，但当前又没检测到清晰门目标时，程序不会傻傻一直往上顶，而是执行一套扫图式搜索动作。

执行逻辑

程序把这个搜索过程看成一个四阶段小状态机。每一阶段都会朝一个固定方向慢慢走，直到位移超过 `amplitude = 20` 像素，就切到下一个阶段。整体轨迹大致是“先向下一点，再向右，再向上，再向右”，形成一种小范围扫描动作。这样做是为了在没有明显上门时，尝试移动到更容易看到或进入上门的位置。

它是干什么的

这不是严格意义上的寻路算法，而是一种启发式搜门手法。它反映了实际刷图中的经验：某些门尤其是上门，角色必须站到特定位置才能顺利接近，所以程序会主动扫一下位置。

片段 15: 通用后备路线移动与位置象限判断

函数 `_fallback_route_move()` 与 `_check_position()`

```
def _fallback_route_move(self, direction: str) -> None:
    normalized = direction.upper()
    if normalized == "DOWN":
        self._reset_route_search()
        if not self._player_xywh:
            Game._action_cache = self._move("DOWN", is_slow=True,
            _action_cache=Game._action_cache)
            return
        player_x = self._player_xywh[0]
        if player_x > self._width * 0.62:
            normalized = "LEFT_DOWN"
        elif player_x < self._width * 0.38:
            normalized = "RIGHT_DOWN"
    elif normalized == "UP":
        self._fallback_up_search_move()
        return
    else:
        self._reset_route_search()

    Game._action_cache = self._move(normalized, is_slow=True,
    _action_cache=Game._action_cache)

def _check_position(self) -> str:
    if not self._player_xywh:
        return "RIGHT_DOWN"
    player_x = self._player_xywh[0]
    player_y = self._player_xywh[1]

    if player_x < self._width / 2:
        return "LEFT_UP" if player_y < self._height / 2 else "LEFT_DOWN"
    return "RIGHT_UP" if player_y < self._height / 2 else "RIGHT_DOWN"
```

这一段的作用

当前画面里如果没有明确目标，比如怪、物品、门都不明显，但导航器又给了一个路线方向，这段代码负责根据这个方向做一个兜底移动。

执行逻辑

如果后备方向是下，程序会根据玩家在屏幕横向位置，适当改成左下或右下，这样更容易撞到真正的下门。若方向是上，则交给前面的“向上扫门模式”。如果方向是左右，则直接慢速朝该方向移动。`_check_position()` 则根据玩家中心落在屏幕哪个象限，返回一个组合方向字符串，后面某些多步方向列表逻辑会用它来判断当前是否已经偏到某个区域。

它是干什么的

这是没有明确目标时的“走位指南”。它并不聪明，但足够实用：至少保证程序在没有门框或门识别不稳时，不会完全原地发呆。

片段 16：主决策入口的开头，玩家定位与丢失恢复

函数 `run()` 的前半段

```
def run(self) -> None:
    raw_player = self._get_cls("player")
    if raw_player is None:
        Game._player_missing_count += 1
        Game._player = Game._pre_player
    else:
        Game._player_missing_count = 0
        Game._player = raw_player
        Game._pre_player = raw_player

    if raw_player is None and Game._player_missing_count >=
Game._missing_player_recover_threshold:
        self._recover_missing_player()
        return

    if Game._player is None:
        return
```

这一段的作用

这是 `Game.run()` 开始决策前的准备工作，核心是在确认玩家当前有没有被检测出来。

执行逻辑

程序先尝试从检测对象里取出当前玩家框。如果这一帧没找到玩家，就把“连续丢失玩家帧数”加一，并且临时沿用上一帧的玩家位置 `_pre_player`，这样可以减少偶发漏检带来的抖动。如果这一帧成功找到玩家，就把计数清零，并更新当前玩家和上一帧玩家缓存。如果连续丢失太多帧，就触发前面说过的恢复逻辑；如果连上一帧缓存也没有，那这一帧就什么都不做。

它是干什么的

动作层任何判断都离不开玩家位置，所以这里相当于先给自己“定位”。而且它考虑到了 YOLO 漏检的现实情况，不会因为一帧没认出玩家就立刻全线失控。

片段 17：更新玩家中心与卡住检测

函数 `run()` 的中段一

```
try:
    self._player_xywh = list(Game._player["xywh"])
    self._player_xywh[0] = self._player_xywh[0] + self._player_xywh[2] / 2
    self._player_xywh[1] = self._player_xywh[1] + self._player_xywh[3] / 2
    logger.info("player center: {}", self._player_xywh[:2])
    self._maybe_update_room_entry()

    if self._update_motion_state(
        expect_motion=Game._action_cache is not None,
        player_visible=raw_player is not None,
    ):
        self._recover_from_stuck("movement_stalled")
        return
```

这一段的作用

这里开始把玩家框换算成玩家中心点，并立刻做“房间入口更新”和“卡住检测”。

执行逻辑

检测器返回的玩家框格式是左上角加宽高，所以这里先把它转换成中心坐标。然后打印日志，接着用这个中心点去更新房间入口信息。做完这些后，再判断当前是不是处于“程序在让角色移动，但角色实际上几乎没动”的卡住状态。如果卡住，立刻执行恢复逻辑并结束这一帧，不再往下做打怪或过门判断。

它是干什么的

这一步体现了一个很合理的优先级：如果角色已经卡住，那无论你后面还要打怪还是进门，先把卡住问题解决掉才有意义。

片段 18：战斗优先级

函数 `run()` 的中段二

```
boss = self._get_clss("boss")
if boss:
    nearest_boss = self._get_nearest(boss)
    if nearest_boss:
        self._kill_monster(nearest_boss["xywh"])
        return

monsters = self._get_clss("monster")
if monsters:
    nearest_monster = self._get_nearest(monsters)
    if nearest_monster:
        self._kill_monster(nearest_monster["xywh"])
        return
```

这一段的作用

这段定义了战斗优先级：如果画面里有 Boss，先打 Boss；没有 Boss，才去打普通怪。

执行逻辑

程序先把所有 Boss 检出来，只要存在，就找最近的 Boss 并调用 `_kill_monster()`。如果这一帧已经进入打 Boss 分支，就直接 `return`，不再继续判断物品和门。若没有 Boss，再对普通怪做相同处理。

它是干什么的

这说明动作层是一个“谁优先谁截断”的设计。也就是说，一帧只做一件最重要的事，不会一边打怪一边又尝试去捡东西或走门。

片段 19: 拾取优先级

函数 `run()` 的中段三

```
goods = self._get_class("goods")
if goods:
    nearest_goods = self._get_nearest(goods)
    if nearest_goods:
        self._pick_up(nearest_goods["xywh"])
    return

money = self._get_class("money")
if money:
    nearest_money = self._get_nearest(money)
    if nearest_money:
        self._pick_up(nearest_money["xywh"])
    return
```

这一段的作用

当这一帧已经没有 Boss 和怪物要处理时，程序会开始考虑拾取地上的物品和金币。

执行逻辑

它先查看是否有一般掉落物 `goods`，有的话找最近的并调用 `_pick_up()`。如果没有普通掉落物，再看是否有金币 `money`。同样，一旦进入拾取分支并执行，当前这一帧就结束，不会继续往下判断门。

它是干什么的

这里体现出刷图行为的整体优先顺序：先清战斗目标，再拾取奖励，最后才考虑过门推进。

片段 20：按路线方向筛门并进门

函数 `run()` 的门处理部分

```
doors = self._get_cls("door")
if doors:
    direction_hint = self._current_door_direction()
    if isinstance(self._direction, (list, tuple)) and Game._index > 0:
        position = self._check_position()
        for item in position.split("_"):
            if direction_hint == item:
                Game._index += 1
                direction_hint = self._current_door_direction()

    usable_doors = self._filter_backtrack_doors(doors, direction_hint)
    nearest_door = self._get_nearest(usable_doors, direct=direction_hint)
if direction_hint else None
    if nearest_door is None and direction_hint:
        logger.info("doors found but none match route hint {}, keep
following hint", direction_hint)
        raise LookupError("no door matches route hint")
    if nearest_door is None and usable_doors:
        nearest_door = self._get_nearest(usable_doors)
if nearest_door:
    self._move_to_door(nearest_door["xywh"])
return
```

这一段的作用

这一段专门负责“清完当前房间之后，按照路线方向去找门并往门走”。

执行逻辑

程序先拿到所有门，再取当前路线方向提示。如果方向本身是个列表，它还会结合当前玩家在屏幕象限，尝试把多步方向列表向前推进。接着先用前面讲过的入口保护逻辑过滤掉明显的回头门，再根据方向提示只找该方向上的最近门。如果明明有方向提示，却完全找不到匹配方向的门，就抛出一个 `LookupError`，表示当前不按错误门，而是把控制权交给后面的兜底逻辑。如果没有严格匹配的门但还有可用门，则退而求其次，选最近的可用门。最终只要拿到门，就调用 `_move_to_door()` 朝它走。

它是干什么的

这是房间级路径真正落地到当前画面行为的关键一段。A 星给的是抽象方向，这里负责把抽象方向落实成“当前这扇门”。

片段 21：异常处理和最终兜底移动

函数 `run()` 的结尾

```
except LookupError:
    pass
except Exception as exc:
    logger.exception("game loop error: {}", exc)
    self._release_cached_action()
    return

fallback_direction = self._current_door_direction()
if fallback_direction:
    if self._should_block_backtrack_direction(fallback_direction):
        logger.warning("fallback direction {} points to entry side, hold position", fallback_direction)
        self._release_cached_action()
        return
    logger.warning("no target found, fallback move: {}", fallback_direction)
    self._fallback_route_move(fallback_direction)
    return

logger.warning("no target found and no route hint, hold position")
self._release_cached_action()
```

这一段的作用

这段代码负责在前面所有主分支都没成功时，决定程序最后该怎么办：是走一个后备方向，还是干脆原地停住。

执行逻辑

如果前面找门时故意抛出了 `LookupError`，这里不会当成严重异常，而是继续走兜底逻辑。只有真正未预料到的异常才会记录日志并释放动作。之后，它再去看有没有路线方向提示。如果有，就先检查这个方向是不是明显指向入口回头边；如果是，就宁可停住也不乱走。否则就调用 `_fallback_route_move()` 做一个带启发式的后备移动。如果连路线提示也没有，那就记录警告并释放按键，保持原地等待。

它是干什么的

这段体现的是动作系统的最后一道安全阀。自动化程序最怕“没想清楚也硬动”，而这里的设计更谨慎：找不到明确目标时，优先做受控的后备动作，实在不行就停。

第七章: `minimap_route_demo.py`

这个文件负责什么： 它不是主刷图脚本，而是一个单独的测试脚本，用来验证“截图 + 大画面检测 + 小地图导航”这一链条是否工作正常。

片段 1：初始化与循环截图

文件前半段

```
import json
import time

import cv2
import numpy as np
import pyautogui

from dnf.detector import Detector
from dnf.minimap_nav import MiniMapNavigator

def main() -> None:
    width = 1067
    height = 600
    detector = Detector("")
    navigator = MiniMapNavigator(map_name="generic")

    while True:
        started = time.time()
        img = pyautogui.screenshot(region=(0, 0, width, height))
        frame_rgb = np.array(img)
        annotated, objects = detector.detect(frame_rgb)
        frame_bgr = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR)
```

这一段的作用

这段脚本做的事情很像 `main.py` 的前半段，但它不是绑定某个游戏窗口，而是直接截屏幕固定区域，然后跑检测和导航。

执行逻辑

先初始化检测器和导航器，然后在循环里每次截屏左上角固定尺寸区域，把截图转换成数组，再分别送给 YOLO 检测器和 OpenCV 导航器。

它是干什么的

它主要是个调试工具，帮助开发者在不真正进入完整刷图主循环的情况下，快速验证导航是否正常。

片段 2：打印导航结果与显示窗口

文件后半段

```
snapshot = navigator.build_route_snapshot(frame_bgr, objects or [])
print(
    json.dumps(
        {
            "current_room": snapshot.current_room,
            "boss_room": snapshot.boss_room,
            "query_room": snapshot.query_room,
            "elite_room": snapshot.elite_room,
            "next_room_direction": snapshot.next_room_direction,
            "selected_door_center": snapshot.selected_door_center,
            "cost_ms": round((time.time() - started) * 1000, 1),
        },
        ensure_ascii=False,
    )
)

if annotated is not None:
    minimap = navigator.extract_minimap(frame_bgr)
    cv2.imshow("dnf-route-frame", cv2.resize(annotated, (800, 450)))
    cv2.imshow("dnf-route-minimap", cv2.resize(minimap, (324, 180)))

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

这一段的作用

这段代码把导航器的结果打印出来，并把检测图和小地图可视化显示给人看。

执行逻辑

程序调用 `build_route_snapshot()` 后，把当前房间、Boss 房、问号房、下一步方向、选中的门中心和耗时都打印成 JSON。然后如果检测图不为空，就显示两扇窗口：一扇是整张大画面的检测结果，一扇是单独裁出来的小地图。按 `q` 时退出。

它是干什么的

这就是一个“路由体检工具”。它不负责按键、不负责刷图，只负责让你肉眼验证导航这一层到底准不准。

附录：为什么这次不逐段讲 `olaplug` 整个目录

原因不是我漏掉了，而是要讲“当前刷图主线真正执行的代码”。`olaplug` 目录里主要是一个非常大的 DLL 封装层，包含窗口绑定、输入、图像、图色、OCR、YOLO 接口、内存接口等大量通用函数。但当前这套刷图主流程并没有真正实例化 `OLAPPlugServer`，也没有调用它的 `BindWindow()`。

补充说明：`olaplug` 在当前项目里的定位

你现在最该知道的，不是它的每个函数，而是它有没有接入主流程

主刷图当前真正走的链路：

`win32gui` 找窗口

- > `pyautogui` 截图
- > `detector.py` 做 YOLO 识别
- > `minimap_nav.py` 识别小地图并调用 A 星
- > `game.py` 用 `pydirectinput` 发方向键 / 攻击键 / 拾取键

而不是：

`OLAPPlugServer.BindWindow()`

- > 后台绑定
- > OLA 输入接口

这一段的作用

这一段是为了防止你读代码时跑偏。`olaplug` 文件很大，接口极多，如果一上来就陷进去，会非常容易把主线看丢。

执行逻辑

当前主刷图代码里真正走到的，是 Windows 找窗口、截图、视觉检测、导航和按键执行这条链。`olaplug` 更像一个备用工具箱，或者旧版本/扩展版本可能会使用的底层能力库。

它是干什么的

如果你之后想深入研究“更底层的后台绑定和 DLL 能力”，我可以下一轮专门单独给你做一本 `olaplug` 说明书。但就“当前刷图代码从头到尾怎么跑”这个目标来说，前面六章才是你最该抓住的主线。

你现在如果按顺序重新回看源码，已经不需要再把它看成一大坨“完全看不懂的代码”了。你可以用一句话串起来：**主程序截图，检测器看大画面，导航器看小地图，A 星给出下一房间方向，门策略把**

方向变成门，动作层再决定是打怪、捡东西，还是朝门走。

文档生成时间：2026-04-23。文档基于当前工作区 `D:\yolo\yolo11\dnf` 的源码内容整理。